

09 - Maximal Independent Set Distribuito

In questa sezione si affronta il problema del calcolo di un'Independent Set massimale in modo distribuito.

Dato un grafo $G = (V, E)$, un suo Independent Set è un sottoinsieme di vertici $M \subseteq V$ tale che non contiene nessuna coppia di nodi adiacenti. Trovare un independent set massimo è noto essere un problema NP-Hard. Un independent set massimale invece è un independent set M il quale non può essere ingrandito ulteriormente con l'aggiunta di un qualsiasi altro nodo in $V \setminus M$, ossia, l'aggiunta di un elemento di $V \setminus M$ ad M fa sì che M non sia più un independent set.

Luby's Maximal Independent Set Algorithm

Si consideri il seguente algoritmo per il calcolo di un independent set.

Algorithm 1: A maximal independent set algorithm

```
Input : A graph  $G = (V, E)$ ;  
Output : A maximal independent set  $M$ ;  
Set  $M := \emptyset$ ;  
while  $V \neq \emptyset$  do  
    Compute an independent set  $S$  in  $G = (V, E)$ ;  
     $M := M \cup S$ ;  
     $V := V \setminus (\Gamma^+(S))$ ;  
     $E := E \setminus \{(u, v) \in E : u \in \Gamma^+(S)\}$ ;  
end  
Return  $M$ ;
```

Dove con $\Gamma(v) \equiv N(v)$ si intende il vicinato del nodo v ; con $\Gamma(S) \equiv N(S)$ si intende l'unione dei vicinati dei nodi in $S \subset V$, formalmente

$$\Gamma(S) = \left(\bigcup_{v \in S} N(v) \right) \cap (V \setminus S)$$

Infine con $\Gamma^+(S)$ si intende S unito $\Gamma(S)$, ovvero $\Gamma^+(S) = \Gamma(S) \cup S$ (analogamente, $\Gamma^+(v) = \Gamma(v) \cup \{v\}$).

L'insieme M ritornato dall'algoritmo è un independent set massimale. Infatti M è sicuramente un insieme indipendente in quanto ogni volta che si uniscono S e M , l'algoritmo rimuove da V tutti i nodi vicini $\Gamma^+(S)$, dunque qualsiasi altro nodo inserito in futuro non avrà vicini in M . Infine M è massimale in quanto ogni nodo in V o viene selezionato per

appartenere ad S , oppure viene rimosso perché appartiene a $\Gamma(S)$. Perciò, aggiungere in M un nodo in $V \setminus M$ creerebbe un arco in M .

Non resta che definire la procedura per calcolare un independent set S ad ogni iterazione. Il calcolo di S è il task che rende questo algoritmo un protocollo, cioè è il task che rende il calcolo di M distribuito.

L'approccio per la definizione di S è il seguente: ogni nodo, all'inizio della iterazione, sceglie un valore di priorità. In particolare, ogni nodo u sceglie un numero u.a.r. in $[R]$ e rende tale numero π_u il suo valore di priorità. A questo punto, ogni nodo u invia π_u ai nodi vicini, cioè in $\Gamma(u)$, e riceve i valori di priorità π_v da ogni nodo $v \in \Gamma(u)$. Dunque, per ogni nodo u , se π_u è il più grande valore di priorità rispetto a quelli ricevuti dai vicini, allora u viene aggiunto ad S . Dunque al termine di questa fase risulta:

$$S = \{u \in V : \forall v \in \Gamma(u), \pi_u > \pi_v\}$$

La probabilità che due nodi vicini abbiano lo stesso valore di priorità è

$$\begin{aligned} \Pr(\exists (u, v) \in E : \pi_u = \pi_v) &= \Pr\left(\bigcup_{(u, v) \in E} \pi_v = \pi_u\right) \\ &\leq \sum_{(u, v) \in E} \Pr(\pi_v = \pi_u) = \frac{|E|}{R} \end{aligned}$$

Allora se $R = n^c$ per $c > 0$ sufficientemente grande, si ottiene che con alta probabilità non ci sono due nodi vicini con lo stesso valore di priorità.

Sulla base di quest'ultima idea, si presenta il protocollo probabilistico per il calcolo di un independent set massimale, noto come *Luby's algorithm*.

Algorithm 2: Luby's maximal independent set algorithm

Input : A graph $G = (V, E)$;

Output : A maximal independent set M ;

Set $M := \emptyset$;

while $V \neq \emptyset$ **do**

Each node v chooses a value π_v from $[n^c]$ uniformly at random;

Let S be the set of nodes such that each of them has bigger priority number than any of its neighbor ($v \in S$ if $\pi_v > \pi_u$ for $u \in \Gamma(v)$);

$M := M \cup S$;

$V := V \setminus (\Gamma^+(S))$;

$E := E \setminus \{(u, v) \in E : u \in \Gamma^+(S)\}$;

end

Return M ;

Analisi del protocollo

Si dimostra che il protocollo di Luby richiede in media $O(\log n)$ iterazioni per terminare. In realtà, si può dimostrare che il protocollo richiede in media $O(\log n)$ iterazioni con alta probabilità, ma questo non verrà dimostrato.

Lemma

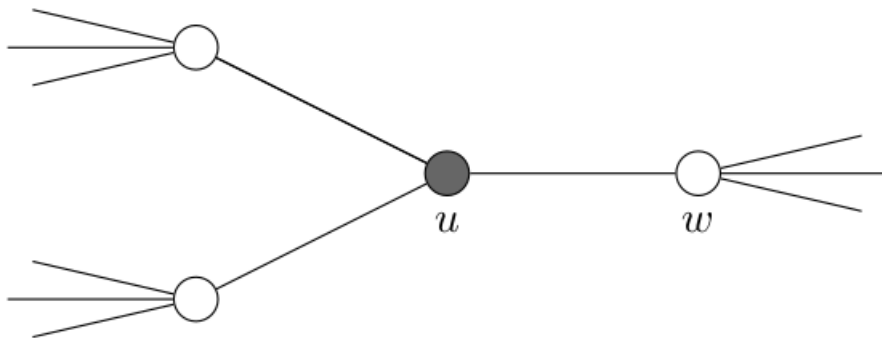
Il numero medio di archi rimossi in una iterazione del protocollo è almeno $\frac{|E|}{2}$, dove E denota l'insieme degli archi del grafo non rimossi nelle iterazioni precedenti.

Dunque ad ogni iterazione, in media, il numero di archi si dimezza. Alla fine, non rimarranno archi in E , verranno inseriti in S tutti i nodi restanti in V e il protocollo termina. Dato che in un grafo ci sono al più $\frac{n(n-1)}{2}$ archi, in media in $O(\log n)$ iterazioni il protocollo termina.

L'idea della dimostrazione è la seguente: si consideri la seguente figura, e sia u un nodo che viene selezionato per appartenere ad S .

Info

I nodi di colore grigio nelle immagini, sono i nodi inseriti in S .

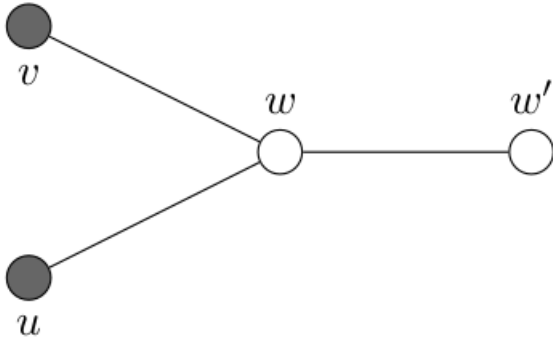


Per definizione del protocollo, sia u che $\Gamma(u)$ vengono rimossi da V e di conseguenza vengono rimossi anche gli archi incidenti ad u e gli archi incidenti ai nodi $\Gamma(u)$. In particolare, si osserva che il numero di archi incidenti ai nodi in $\Gamma(u)$ è almeno il numero di archi incidenti ad u : sono proprio gli archi che rendono i nodi in $\Gamma(u)$ dei vicini di u .

Dimostrazione

Sia, in una generica iterazione dell'algoritmo, $u \in S$ e $w \in \Gamma(u)$. Allora gli archi incidenti a w verranno rimossi alla fine dell'iterazione, in quanto u viene selezionato e inserito in S . Inoltre, potrebbe esistere un altro nodo $v \in S$ tale che $w \in \Gamma(v)$. Allora è corretto dire che gli archi

incidenti a w verranno rimossi in quanto $v \in S$. Si rappresenta tale scenario nella seguente figura



Allora l'arco (w, w') viene rimosso sia per colpa di u , che per colpa di v . Ciò nonostante, per una corretta analisi si deve contare la rimozione di (w, w') una sola volta.

Per definizione, si dirà che " x rimuove y " se $x \in S, y \in \Gamma(x)$ e

$\pi_x > \pi_z, \forall z \in \Gamma(x)^+ \cup (\Gamma(y)^+ \cap S)$, cioè se x ha il più alto valore di priorità non solo tra tutti i suoi vicini, ma anche tra tutti i vicini del nodo y che appartengono ad S . Sia in questo caso, senza perdita di generalità, che " u rimuove w ", cioè che $\pi_u > \pi_v$ (si osserva che $w' \notin S$).

Ma l'arco (w, w') potrebbe essere rimosso anche perché esiste un $u' \in S$ tale che " u' rimuove w' ": allora si conta anche questa rimozione di (w, w') (che verrà opportunamente eliminata dal conteggio alla fine): l'importante è non contare tale rimozione più di due volte.

La probabilità che " u rimuove w " è pari alla probabilità che π_u sia il più grande valore di priorità tra tutti i nodi in $\Gamma(u)^+ \cup (\Gamma(w)^+ \cap S)$.

Sia $X_{u \rightarrow w}$ una variabile aleatoria binaria che vale 1 se l'arco (w, w') è rimosso per colpa di u (cioè se " u rimuove w ") e vale 0 altrimenti. Allora vale che

$$\begin{aligned} \mathbf{E}[X_{u \rightarrow w}] &= \mathbf{Pr}(X_{u \rightarrow w} = 1) = \frac{1}{|\Gamma(u)^+ \cup (\Gamma(w)^+ \cap S)|} \\ &\geq \frac{1}{|\Gamma(u)^+ \cup \Gamma(w)^+|} \\ &\geq \frac{1}{d(u) + d(w)} \end{aligned}$$

dove $d(\cdot)$ indica il grado del nodo, e l'ultima disuguaglianza è data dal fatto che nel caso peggiore i due vicinati hanno intersezione vuota, cioè u non condivide vicini con w .

Allora, se " u rimuove w ", $d(w)$ è il numero archi adiacenti a w rimossi per colpa del nodo u . Dunque il valore atteso del numero di archi rimossi nell'iterazione perché " u rimuove w " per ogni $u, w \in V$, sapendo che la rimozione di ogni arco viene contata al più due volte, è pari a

$$\begin{aligned}
\frac{1}{2} \sum_{u,w \in V: (u,w) \in E} \mathbf{Pr}(X_{u \rightarrow w} = 1) d(w) &= \frac{1}{2} \sum_{(u,w) \in E} \mathbf{Pr}(X_{u \rightarrow w} = 1) d(w) + \mathbf{Pr}(X_{w \rightarrow u} = 1) d(u) \\
&\geq \frac{1}{2} \sum_{(u,w) \in E} \frac{d(w)}{d(u) + d(w)} + \frac{d(u)}{d(u) + d(w)} \\
&= \frac{1}{2} \sum_{(u,w) \in E} \frac{d(u) + d(w)}{d(u) + d(w)} \\
&= \frac{1}{2} \sum_{(u,w) \in E} 1 \\
&= \frac{|E|}{2}
\end{aligned}$$